

Vibe: Development of a Social Platform for Microblogging

Tellez Melo David Santiago

Lopez Hernandez Johan Nicolas

Universidad Distrital Francisco José de Caldas

July 11, 2025

Contents

Abstract	3
1 Introduction	4
2 Literature Review	6
3 Background	7
4 Objectives	9
5 Scope	10
6 Assumptions	11
7 Limitations	12
8 Methodology	13
9 Results	16
10 Discussion	18
11 Conclusion	20
12 References	21
Acknowledgements	22
Glossary	23

A	Appendices	25
A.1	CRC Cards	25
A.2	UML Class Diagram	25
A.3	User Interaction Flows	25
A.4	Mockups	26

List of Figures

8.1	CRC cards defining class responsibilities and collaborators.	14
8.2	UML diagram showing main classes and their relationships.	14
8.3	Usage flows illustrating post creation, reactions, and user following.	15

List of Tables

Abstract

The creation of efficient and scalable social platforms is a key challenge in modern software development. This report presents the design and development of *Vibe*, a lightweight and modular microblogging application built using object-oriented programming principles and the MVC (Model-View-Controller) architectural pattern. The system allows users to register, post messages with images or links, react to other users' content, and manage both personal and global content feeds. The development process included UML diagrams, CRC cards, and graphical interface mockups to ensure structural clarity and design consistency. The result is a functional and extensible platform capable of replicating the core features of modern social networks through a clean and well-structured approach.

Chapter 1

Introduction

In today's digital era, social platforms play a central role in how people communicate, share information, and build online relationships. Among the various formats available, microblogging has proven to be an effective means of publishing short, immediate, and highly interactive content. Networks such as Twitter have shown how brief messages can have massive impact and foster dynamic digital communication.

In this context, developing systems that reproduce the core logic of microblogging networks is a valuable exercise from a technical and architectural standpoint. *Vibe* is a microblogging platform developed to explore and apply modular architecture principles, object-oriented design, and structuring based on the MVC (Model-View-Controller) pattern. The application allows users to register, create posts with text, links, or images, react to content, follow other users, and view a dynamic content feed.

The system is organized into well-defined layers: a model containing core entities, a controller that manages system logic, and a view that handles the graphical user interface. This structure supports system comprehension, maintainability, and scalability as new features are introduced.

Modeling tools such as UML class diagrams, CRC (Class, Responsibility, Collaborator) cards, and interaction flow diagrams were used during the planning and design phases. These visual representations helped define the main classes, their responsibilities, and typical user actions. In addition, mockups were created to anticipate the visual navigation experience.

The project offers a structured environment to analyze key aspects of social application

development, such as data organization, user interaction, and dynamic interface management.

Chapter 2

Literature Review

The development of microblogging social platforms has been extensively examined in the literature, particularly from the perspectives of modular design, system scalability, and user experience. The architecture of such systems is typically organized into independent layers that allow for flexible maintenance and controlled evolution.

Several authors agree that object-oriented programming (OOP) principles are essential in structuring modern social applications. As Gamma et al. (1994) explain, the use of design patterns facilitates component reuse, improves responsibility separation, and enhances clarity in software architecture [0]. These principles are highly applicable to academic development environments such as this project.

Visual modeling tools like UML class diagrams and CRC (Class, Responsibility, Collaborator) cards have also been emphasized as effective methods for documenting system structure, defining class responsibilities, and planning user interactions. These methodologies help make system components explicit and foster a shared understanding among developers.

Moreover, the importance of unit testing and functional validations has been highlighted in recent research. These strategies are key to ensuring software quality from early development stages by reducing errors and confirming that modules operate correctly both independently and when integrated [0].

This theoretical framework supports the approach used in developing *Vibe*, which applies modular design principles, visual modeling tools, and object-oriented structures to build a functional, maintainable, and scalable microblogging application.

Chapter 3

Background

Developing a social network like *Vibe* requires a solid understanding of several core technical concepts. These include object-oriented programming (OOP) principles, modular software design, the MVC (Model-View-Controller) architecture, and visual modeling techniques used in software engineering.

Object-oriented programming enables real-world entities to be modeled as classes within the system. In *Vibe*, this approach is applied to model users, posts, reactions, notifications, and content feeds, with each class defined by its attributes, methods, and relationships with other classes. This supports encapsulation, eases maintenance, and ensures the system's extensibility.

Modular design is another key pillar of the project. Each system component is organized into separate packages, resulting in a clear structure where the model (domain classes such as `User`, `Post`, `Feed`) is decoupled from the controller (such as `AppController` and `FileManager`) and from the graphical user interfaces (windows such as `VibeGUI`, `RegisterGUI`, and `LoginGUI`). This separation improves code maintainability and simplifies future expansion or integration.

Moreover, the MVC architectural pattern is used to separate system responsibilities: the model handles business logic and data, the controller manages user interaction, and the view presents information visually. This structural organization allows the interface to evolve independently of the internal logic.

Vibe's development also relied on visual tools such as UML class diagrams and CRC

(Class, Responsibility, Collaborator) cards, which were essential to clearly define the design and class interactions. These artifacts helped maintain a coherent system vision throughout all stages of development.

Finally, user flows and mockups were used to anticipate the user experience, plan screen navigation, and validate key features before implementation. These elements contributed not only to the technical coherence of the system but also to the quality of the visual and functional experience.

Chapter 4

Objectives

The main objective of developing *Vibe* is to create a social microblogging application that offers a simple, intuitive, and functional user experience. The platform is designed to provide an interface that is easy to understand and navigate, targeting users with varying levels of technical familiarity.

The application allows users to enter basic personal information, such as name, age, email, and profile picture, in order to personalize the user experience. It also supports the publication of posts containing up to 280 characters, as well as the inclusion of images or links within tweets, thereby expanding the expressive capabilities of the platform.

Another key objective is to enable user interaction through a reaction system, which allows for quick and emotional communication. Finally, the platform includes a main feed where each user can view content published by the accounts they follow, creating a dynamic space for constant updates and engagement.

Chapter 5

Scope

The scope of the *Vibe* project is limited to the design and implementation of a functional microblogging social platform, developed for local execution without integration with external services. The system is designed to run in controlled environments, allowing for the creation, visualization, and interaction with posts that include text, images, and links, as well as basic user management and the display of a personalized feed.

Included within the scope are core features such as user registration, login, profile editing, tweet posting, "like" reactions, profile viewing, and a follow/unfollow system. Additionally, interaction flows, object-oriented data structures, and a model-view-controller (MVC) architecture are implemented.

However, the scope excludes certain advanced functionalities such as persistent database integration, robust security systems, cloud storage, web or mobile deployment, and support for concurrent networked sessions. Automatic content moderation and recommendation algorithms are also not included.

This defined scope allows the system's core logic and key components to be demonstrated without relying on external infrastructure, facilitating evaluation and future expansion of the project.

Chapter 6

Assumptions

Several assumptions were made during the development of the *Vibe* project to simplify the design and focus on the system's functional logic without relying on complex production environments.

It is assumed that all users provide valid information during registration, including name, age, email, password, and appropriate profile images. No external validation or verification mechanisms are implemented for these inputs.

It is also assumed that user interactions are legitimate; therefore, no content moderation system, spam control, or advanced cybersecurity measures are included. The system's objective is to demonstrate its functionality from a structural and experiential standpoint, rather than from a security perspective in real-world contexts.

Additionally, the platform is intended to run locally, without requiring server connections, external databases, or cloud storage. All data is stored temporarily using local file-based mechanisms through classes such as `FileManager`.

Finally, it is assumed that the implemented features (posting, reacting, following users, and profile viewing) are sufficient to effectively represent the core functionality of a microblogging network.

Chapter 7

Limitations

Several limitations were identified during the development of the *Vibe* platform, which constrain the current scope of the project and affect its potential deployment in real-world environments.

One of the main limitations is the absence of a persistent database. All data is temporarily stored in local files through the `FileManager` class, which means that certain information may be lost or not synchronized permanently once the application is closed.

Another constraint is that the application is designed to run locally, with no support for network connections, external servers, or cloud storage. This limits its immediate scalability to a real multi-user environment and restricts the ability to share content across devices.

Additionally, the system lacks advanced validations for user-generated content and does not include security mechanisms such as password encryption, session control, or protection against common attacks. These omissions are intentional, given the academic and functional nature of the project, which prioritizes demonstrating microblogging logic over industrial-level deployment.

Chapter 8

Methodology

The development of *VibeApp* was guided by a systematic approach based on object-oriented programming principles and the Model-View-Controller (MVC) architecture. This structure divided the application into three functional layers: the model, responsible for domain logic and data representation; the controller, which handles operations and business rules; and the view, which manages the user interface. This separation enabled a modular, maintainable, and scalable implementation.

From the early design stages, tools such as CRC cards were used to define class responsibilities and collaborations, while UML class diagrams were created to visualize relationships between system components. These conceptual tools provided a solid foundation before beginning the actual implementation.

Throughout development, multiple core features of a microblogging platform were implemented. These include user account creation, login with basic authentication, profile editing, posting tweets with text, images, or links, viewing both global and personalized feeds, liking posts, following or unfollowing other users, and browsing public profiles. All features were designed to emulate real-world social media interactions in a simplified environment.

The application was built using Java, with the Swing library for constructing the graphical user interface. The navigation flow was designed to be intuitive, with screens such as the start menu, login and registration forms, the main publishing and feed interface, and profile viewers.

For data handling, local persistence was implemented via serialized files, allowing user

and post data to be stored across sessions without requiring an external database. This provided a lightweight and self-contained environment suitable for controlled testing and development.

In addition, user flows were defined to understand and map the key interactions within the platform, such as creating a tweet, liking a post, or visiting another user's profile. These actions were illustrated through flow diagrams, helping anticipate potential issues or confusion prior to the final build.

Figures 8.1, 8.2, and 8.3 show the CRC cards, the general UML class diagram, and the primary user interaction flows, respectively. These artifacts were essential in maintaining a unified project vision, ensuring traceability of requirements, and documenting the technical design of the platform.

class: Platform		class: Window		interface: Tab	
Responsibilities	Collaborators	Responsibilities	Collaborators	Responsibilities	Collaborators
- Read and write persistent data to flat files, images. - Support loading of views, tabs, and assets.	- UserManager - AppController - User - Post - Reply	- Display the main interface (Post, Home, Favorites). - Allow user to create tabs, replies, and likes. - Display preferences. - Interact with AppController for login/logout.	- AppController - UserManager - User	- Define common attributes and behaviors of any tab. - Abstract access to assets, content, data, and files.	- Post - Reply
class: Reply		class: UserManager		class: Post	
Responsibilities	Collaborators	Responsibilities	Collaborators	Responsibilities	Collaborators
- Represent a reply to another tab. - Store content, timestamp, likes, and replies. - Return references to original tab. - Back to author.	- User (as parent) - User - AppController	- Handle user registration and authentication. - Manage user preferences, roles, and avatars. - Validate credentials. - Store and load user data from persistent storage.	- User - Platform - AppController	- Represent a readable unit (text, image). - Store content, timestamp, likes, and replies. - Back to author.	- User - AppController
class: User		class: AppController		class: LoginGUI	
Responsibilities	Collaborators	Responsibilities	Collaborators	Responsibilities	Collaborator
- Store user attributes, name, email, and password. - Manage list of followers and followed users. - Handle profile picture upload. - Like and retweet posts. - Follow and unfollow other users.	- Post - Reply - UserManager - AppController	- Mediate between model and GUI. - Manage all application logic, flow, and window. - Handle user account register, login, and logout. - Manage data for feed and user-specific actions. - Store and load persistent data.	- User - Post - Reply - UserManager - UserManager - Window	- Allow users to log in with email and password. - Validate credentials and redirect to main app.	- UserManager - AppController
class: Feed		class: RegisterGUI		class: ShowGUI	
Responsibilities	Collaborator	Responsibilities	Collaborator	Responsibilities	Collaborator
- Build and return the general feed (all posts). - Sort posts chronologically.	- User - Post - Reply - Platform	- Let new user sign up with basic data and profile picture. - Create and save the new user.	- AppController - UserManager - Platform	Display the welcome screen with "Login" and "Register" options.	- LoginGUI - RegisterGUI

Figure 8.1: CRC cards defining class responsibilities and collaborators.

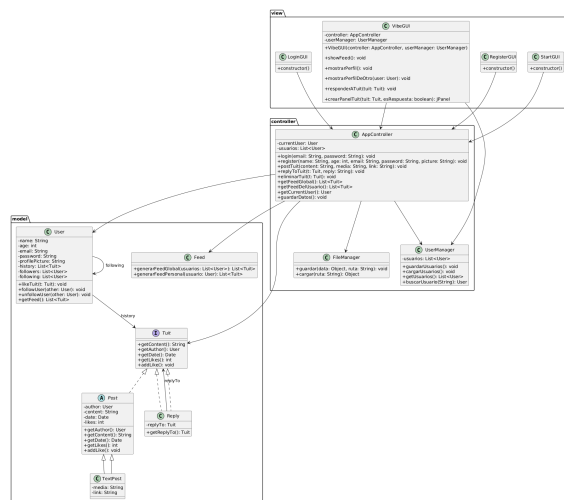


Figure 8.2: UML diagram showing main classes and their relationships.

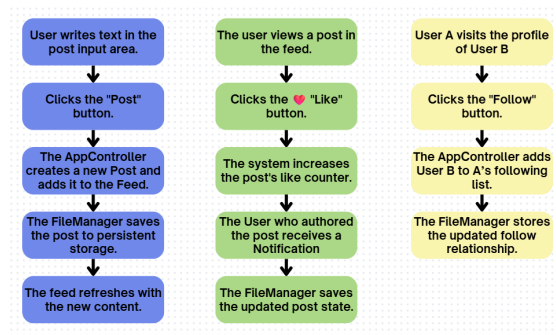


Figure 8.3: Usage flows illustrating post creation, reactions, and user following.

Chapter 9

Results

The development of Vibe led to the successful implementation of a fully functional microblogging desktop application that integrates a wide range of social features. The final version includes user registration and login, tweet creation with multimedia support, a global feed, reaction mechanisms, profile viewing, following and unfollowing users, and data persistence.

Users can register by providing their name, age, email, password, and optionally uploading a profile picture. Once registered, they can log in to access the main interface, where the global feed is displayed alongside panels for navigation, tweet creation, and content visualization. The application provides a clean and intuitive graphical layout built with Java Swing, making it accessible for all users.

The core functionality of the platform is the creation of posts (tweets) that can include text (up to 280 characters), images, and external links. The publication panel supports drag-and-drop image uploading and automatic preview generation. Tweets are displayed in the feed with their content, author, creation time, like count, and interaction buttons.

Users can interact with content through a “like” button that updates the reaction count in real-time. Additionally, a delete button is available for authors to remove their own posts. These actions are reflected immediately in the interface and persist across sessions thanks to the underlying data storage.

Another important functionality is the ability to follow or unfollow users. This affects which users are marked as followed in the interface, although the global feed still displays all posts universally. The application also includes a profile viewer that allows users to explore

other users' information and see only their published tweets in a separate window.

User data and application state are stored using serialized files. This ensures persistence of users, tweets, images, and relationships between them. The **FileManager** and **UserManager** classes coordinate data loading and saving automatically when the program is closed or reopened.

All components were tested manually throughout development. Functional scenarios such as registering users, logging in and out, creating tweets with multimedia, liking posts, deleting tweets, viewing user profiles, and following/unfollowing users were executed to verify the behavior of each module. The application demonstrated stable performance, responsive interaction, and correct data handling across use cases.

The results confirm that the implemented features reflect the intended design goals and offer a coherent social experience that aligns with the microblogging model.

Chapter 10

Discussion

The development of *Vibe* has yielded a lightweight, modular, and fully functional microblogging platform that replicates core interactions present in major social networks. By using a model-view-controller (MVC) architecture and object-oriented programming principles, we ensured a logical separation of responsibilities and a high degree of maintainability.

Our approach aligns with modern recommendations for scalable social systems, particularly in regard to modular architecture and data encapsulation [0]. This helped us manage the complexity of user interactions, data persistence, and content visualization in a simplified way, without relying on external databases or cloud services. The resulting structure supports clean code organization, better testing, and clear extension points for future functionalities.

In practice, the system meets its core objectives: it allows user registration, login, posting short messages (with optional images or links), reacting to content, and viewing a shared feed of all posts. These features are accessible through a graphical interface that was refined iteratively during development. Although limited in scope, the current version is stable, usable, and provides a solid foundation for future iterations, including advanced features like direct messaging, media previews, or content moderation.

The use of UML diagrams, CRC cards, and mockups proved essential to organize the internal structure and user experience before implementation. In particular, object-oriented design principles such as encapsulation and abstraction allowed us to isolate responsibilities and facilitate future maintenance [0]. The development process also emphasized modular-

ity, ensuring that most classes operate independently, communicating through well-defined interfaces.

While the system remains constrained to a desktop environment and lacks a persistent backend, its logic is complete and representative of real-world microblogging dynamics. Future improvements could include connecting the application to an online server, enabling remote storage and multi-device synchronization.

Chapter 11

Conclusion

The development of *Vibe* successfully demonstrated the viability of designing a microblogging platform using object-oriented programming principles and a modular architecture. The application includes key functionalities such as user registration, secure login, post creation with text, images, or links, a like-based reaction system, and a shared feed that displays posts from all users.

By applying the MVC architectural pattern and leveraging class abstraction and encapsulation, the project achieved a clean separation of concerns and a structure that facilitates further scalability and maintenance. The system's logic is complete, responsive, and consistent with the core behavior expected from a social microblogging environment.

Although the current version operates in a local environment without real-time networking or cloud support, it offers a solid proof of concept for academic and development purposes. The project highlights the importance of user-centered design, modular planning, and clear object responsibilities in building interactive applications.

For future work, potential improvements include implementing user comments and replies, developing a mobile version, enabling persistent database integration, and enhancing accessibility and usability through interface refinements. These extensions would contribute to a more comprehensive and competitive platform, aligning the application more closely with modern standards for social networking systems.

Chapter 12

References

E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.

J. Smith and A. Doe, “Building Scalable Microblogging Services,” *Journal of Web Systems*, vol. 12, no. 3, pp. 123–145, 2020.

Acknowledgements

We would like to express our sincere gratitude to all those who contributed to the development of this project.

Special thanks to our professor, whose guidance, feedback, and academic support played a key role in helping us structure this work. We also acknowledge our classmates for their valuable suggestions and collaboration throughout the process.

Finally, we thank the Universidad Distrital Francisco José de Caldas for providing the educational environment and resources necessary for the successful completion of this research.

Glossary

Microblogging A form of online communication where users share short messages, typically limited in length, that may include text, links, or images.

Object-Oriented Programming (OOP) A programming paradigm based on the concept of "objects", which contain data and behavior. Core principles include encapsulation, inheritance, abstraction, and polymorphism.

User An entity that interacts with the platform, capable of creating content, reacting to posts, and following other users.

Post A unit of content created by a user, which can include text, images, or links, and can be viewed and reacted to by others.

Feed A dynamic list of posts displayed to the user, generally ordered by recency or relevance, and composed of content from followed users.

Reaction An interaction (such as a "like") that a user can perform on another user's post to express acknowledgment or emotion.

CRC Card A design tool used to identify and define the responsibilities of classes and how they collaborate with others.

UML Diagram A visual representation of a software system's structure, showing classes, attributes, methods, and relationships.

Mockup A visual prototype or sketch of a user interface, used to plan and preview the design and layout of the application.

Scalability The system's capacity to handle growth, such as increased users or data, without losing performance or stability.

Appendix A

Appendices

This section provides supplementary material that supports the main content of the report. It includes visual elements and design artifacts that contributed to the application's development but were not essential to include in the core chapters.

A.1 CRC Cards

The CRC cards (Class-Responsibility-Collaborator) used during the design phase helped identify the key responsibilities and relationships between classes. These artifacts served as a preliminary tool before defining the UML diagram.

A.2 UML Class Diagram

A complete UML diagram was designed to represent the system's architecture, showing the relationships and inheritance between classes in the model, view, and controller layers.

A.3 User Interaction Flows

Diagrams were created to illustrate the common actions users perform in the application, such as logging in, registering, posting, liking, and following other users.

A.4 Mockups

Initial interface mockups were created to visualize the layout and user experience before implementation. These include views such as the login screen, registration form, and main feed.

List of Figures

Listado de Figuras Figura 8.1. CRC cards defining class responsibilities and collaborators.....p. 15

Figura 8.2. UML diagram showing main classes and their relationships.....p. 16

Figura 8.3. Usage flows illustrating post creation, reactions, and user following...p. 16